

Assignment Report

The Assignment report submitted in partial fulfilment of the
requirements for

the award of the degree of

The Professional Master Of Embedded And Cyber-Physical Systems



ECPS 216 - F'24: IOT Systems & Software

Programming Assignment - 5

{ Code RED }

Under the guidance of

Prof. Quoc-Viet Dang

Name of Candidates

Sahil Kakadiya

Vaibhav Gajera

Rohankumar Barouliya

UCI Student ID

93282494

89125790

79491905

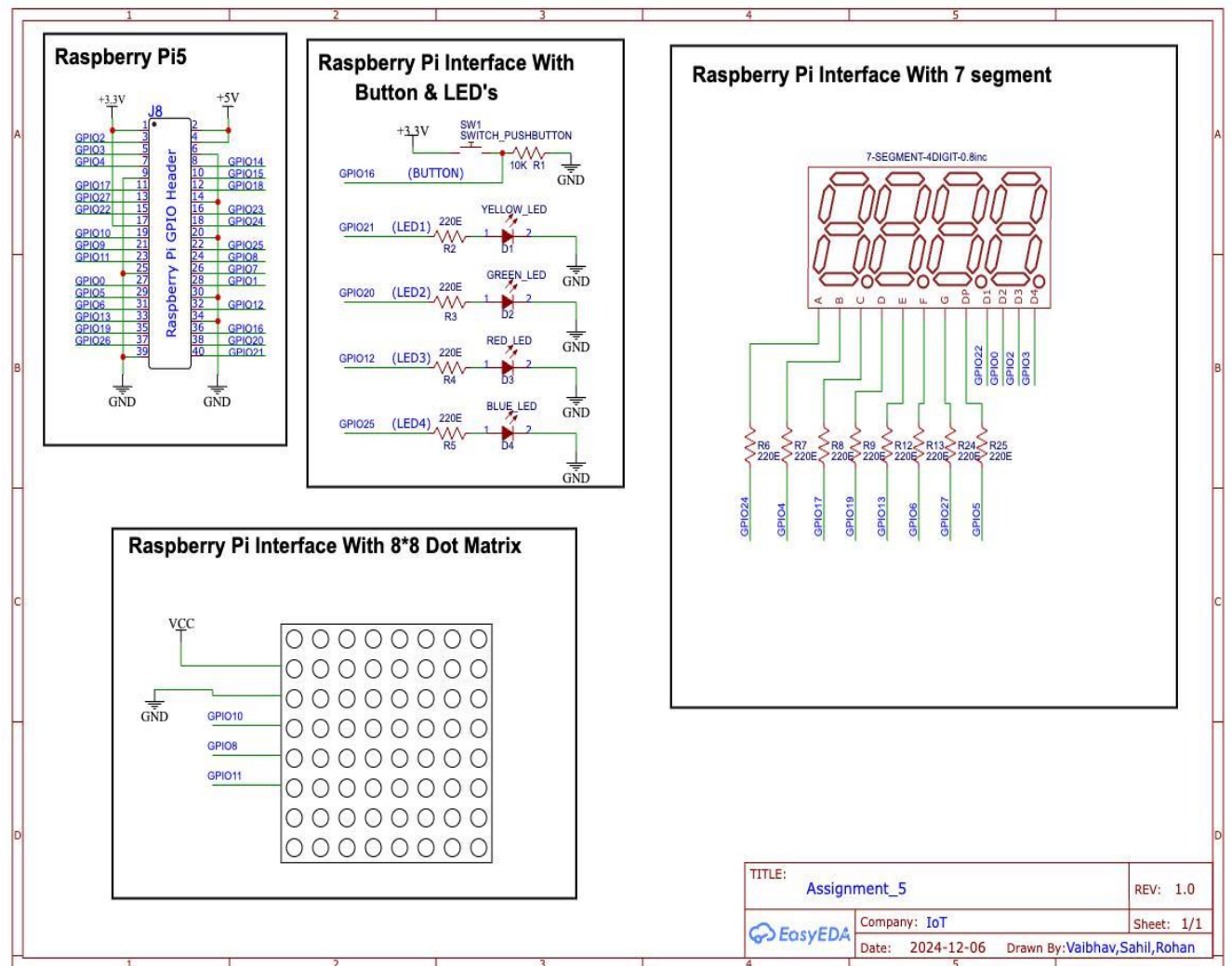
Assignment-5 Code RED

➤ Google Drive Demo Link:

https://drive.google.com/file/d/1rmch__JR521w0odVEQ8Vy65QuiAksdOP/view?usp=sharing

Overview - Schematic and Communication Protocol

➤ Schematic Of Raspberry Pi



(A.) Raspberry Pi Schematic

This schematic demonstrates the interfacing of a Raspberry Pi with various electronic components, including buttons, LEDs, a 7-segment display, and an 8x8 dot matrix display. Below is a detailed explanation of each section and the connections:

1. Raspberry Pi GPIO Header (J8)

Purpose: Acts as the central control unit for the circuit.

Description: The header represents the GPIO pins of the Raspberry Pi. These pins provide input/output functionality, ground (GND), and power (+3.3V, +5V).

Connections: These pins are connected to the respective components:

- ➔ GPIO16, GPIO20, GPIO21, GPIO12, GPIO25 for button and LEDs.
- ➔ GPIO22, GPIO24, GPIO5, GPIO6, GPIO13, GPIO19, GPIO26, GPIO18 for the 7-segment display.
- ➔ GPIO10, GPIO8, GPIO11 for the 8x8 dot matrix.

2. Raspberry Pi Interface with Buttons & LEDs

Components:

Button (SW1):

Connected to GPIO16 through a 10k Ω pull-up resistor.

Acts as an input signal to Raspberry Pi.

LEDs:

Yellow (LED1) - GPIO21.

Green (LED2) - GPIO20.

Red (LED3) - GPIO12.

Blue (LED4) - GPIO25.

Resistors:

Each LED is connected to a 220 Ω resistor to limit current and prevent damage.

Working:

Pressing the button sends a signal to GPIO16.

Raspberry Pi can control the LEDs by toggling their respective GPIO pins.

3. Raspberry Pi Interface with 7-Segment Display

Component: 4-digit, common cathode 7-segment display.

GPIO Connections:

➔ GPIO22, GPIO24, GPIO5, GPIO6, GPIO13, GPIO19, GPIO26, GPIO18.

Resistors:

Each segment is connected through a 220 Ω resistor for current limiting.

Working:

The Raspberry Pi controls the segments of the 7-segment display using GPIO pins. By lighting specific segments, numerical or alphanumeric characters can be displayed.

4. Raspberry Pi Interface with 8x8 Dot Matrix

Component: 8x8 dot matrix LED display.

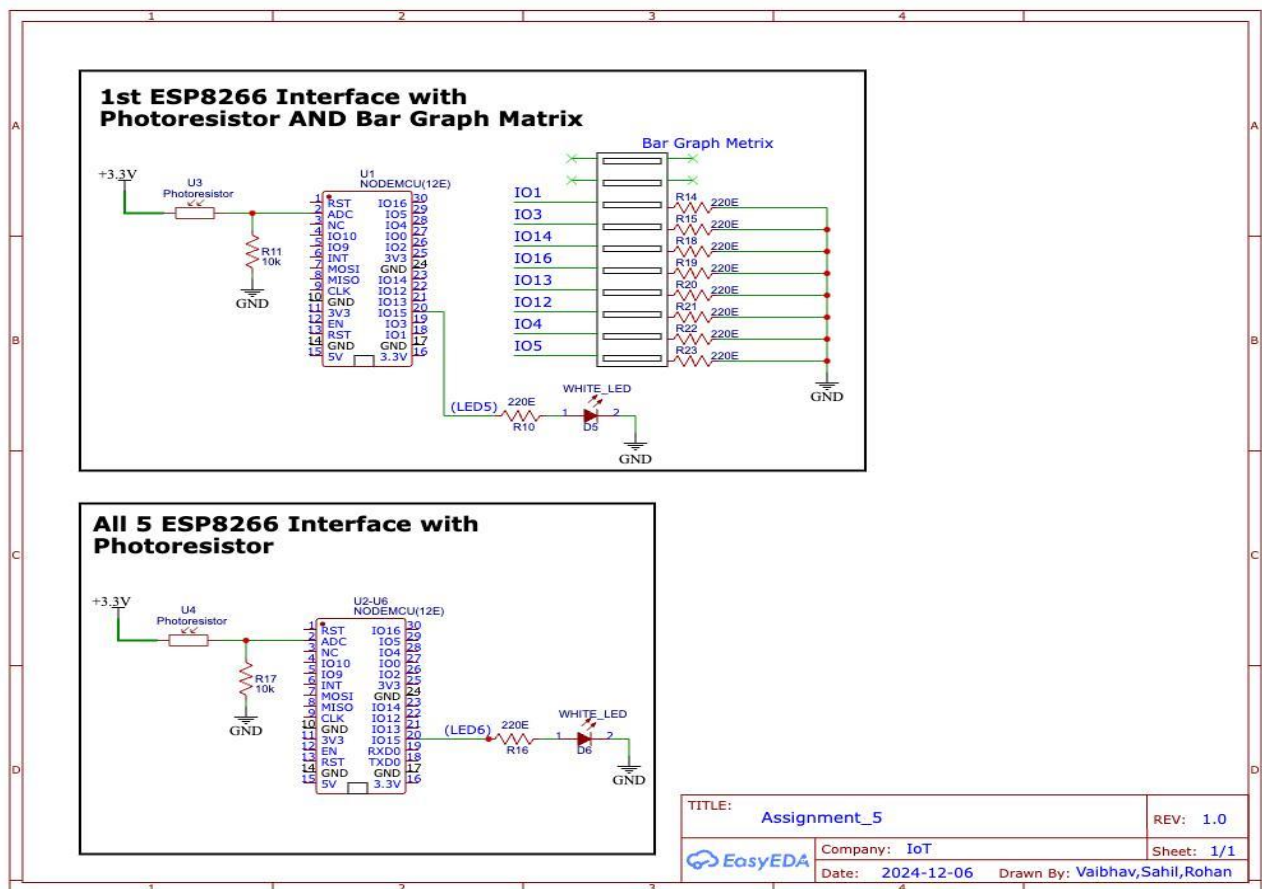
GPIO Connections:

➔ GPIO10, GPIO8, and GPIO11 are used to drive the rows/columns.

Working:

The Raspberry Pi sends signals to specific rows and columns to light up individual LEDs in the matrix. Patterns or characters can be displayed by controlling which LEDs are turned on.

➤ Schematic Of Raspberry Pi



This schematic illustrates the interfacing of ESP8266 NodeMCU microcontrollers with photoresistors, bar graph displays, and LEDs. Here's a detailed explanation of the setup:

1. First ESP8266 Interface with Photoresistor and Bar Graph Matrix

This section demonstrates the connection of an ESP8266 NodeMCU to:

A photoresistor for light sensing. A bar graph matrix for visual representation of the measured values. A white LED for signalling or illumination.

Connections:

Photoresistor (U3):

Connected to ADC (Analog to Digital Converter) pin of the NodeMCU. The photoresistor is connected in a voltage divider configuration with a 10kΩ resistor (R11) to ground.

Function: Measures the intensity of light and provides an analog voltage output that is read by the ADC pin.

Bar Graph Matrix:

The matrix consists of multiple LEDs controlled via GPIO pins:

IO3, IO14, IO16, IO13, IO4, IO5, etc.

Each LED in the bar graph is connected through a 220Ω resistor (R14, R15, ... R23) to limit the current.

Function: Displays a bar graph representation of the photoresistor's light intensity readings.

White LED (LED5):

Connected to IO5 through a 220Ω resistor (R10).

Function: Provides a binary visual signal or illumination.

2. All 5 ESP8266 Interface with Photoresistor

This section extends the setup to multiple ESP8266 NodeMCU microcontrollers (U2–U6), each interfaced with:

A photoresistor for individual light intensity measurements.

A white LED for signaling.

Connections:

Photoresistors:

Each NodeMCU has its own photoresistor connected to the ADC pin.

A 10k Ω resistor (R17) is used as part of the voltage divider.

Function: Each photoresistor measures light intensity independently.

White LED:

Connected to the GPIO pin of each NodeMCU via a 220 Ω resistor (R16).

Function: Acts as an indicator or alert based on the light intensity measured.

➤ Communication Protocol used

1. Communication Protocol:

- The system uses **UDP (User Datagram Protocol)** for communication.
- UDP is lightweight and designed for **real-time, low-overhead communication**.

2. Broadcast Mechanism:

- Each ESP8266 broadcasts its **sensor readings** to a **universal IP address (255.255.255.255)**.
- This enables all devices in the swarm to **listen and respond** without needing specific IP addresses.

3. Simplified Network Setup:

- The broadcast capability of UDP makes network setup easier.
- Devices can communicate without requiring predefined IP addresses or complex configurations.

4. Dynamic Master Selection:

- The system supports **dynamic Master selection** based on sensor readings.
- The ESP8266 with the **highest reading** becomes the **Master** and directly communicates with the Raspberry Pi.

5. Real-Time Responsiveness:

- UDP enables **fast data exchanges** with minimal delay.
- Its lightweight nature is crucial for **real-time responsiveness** in IoT systems.

6. Connectionless Nature:

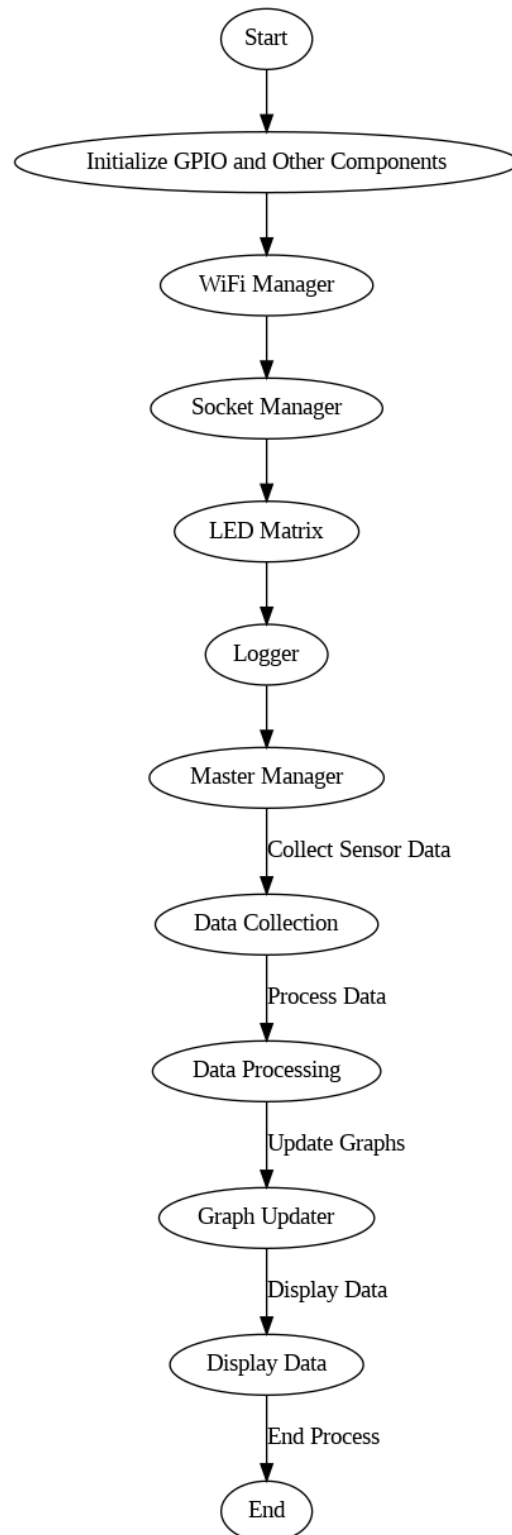
- UDP is **connectionless**, meaning there is no need to establish a connection before communication.
- Devices can **join or leave** the network freely without affecting overall operation.

7. Flexibility and Adaptability:

- The connectionless and broadcast features of UDP make it ideal for a **flexible and adaptive IoT system**.
- The system can handle changes in the swarm dynamically.

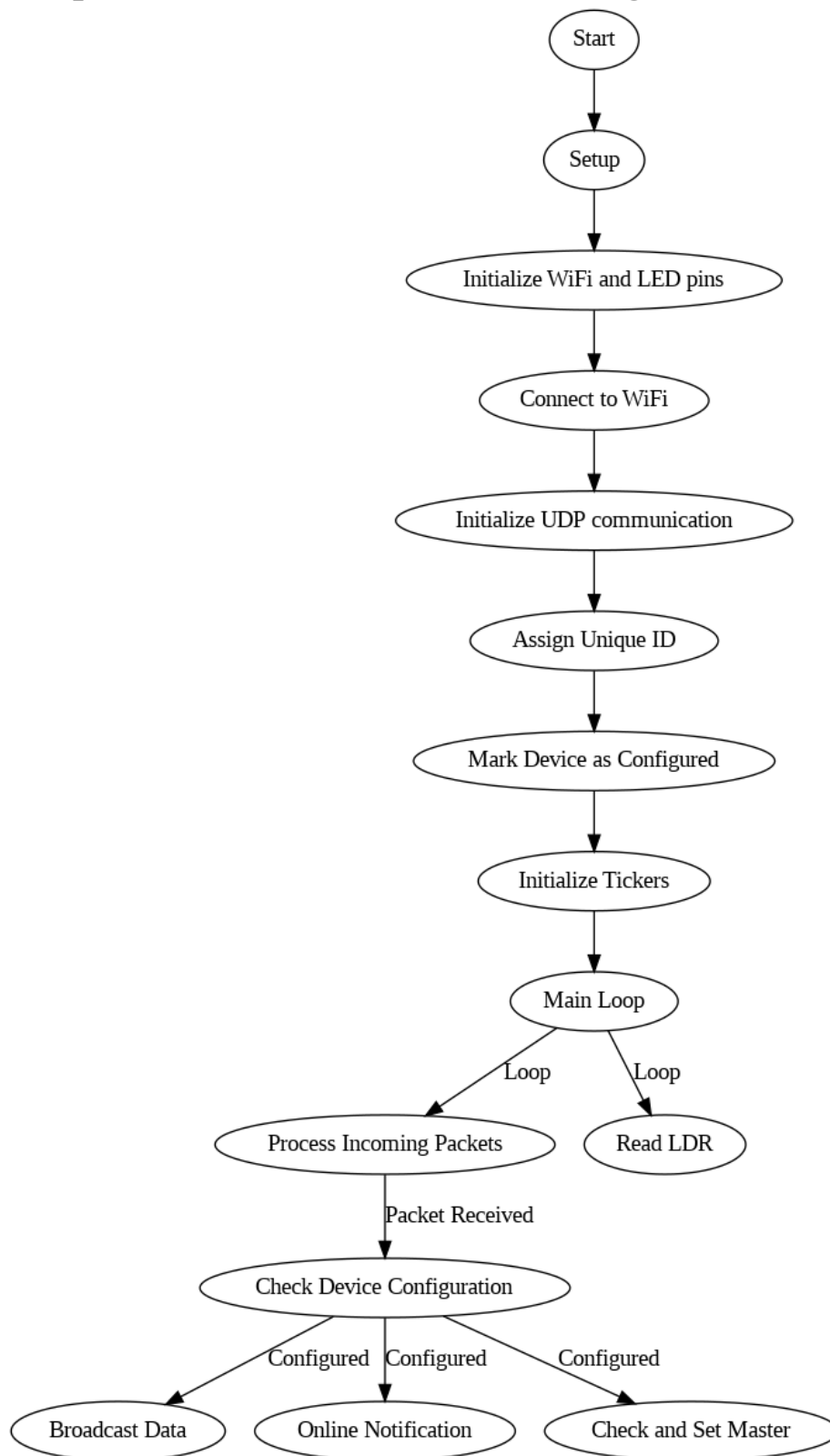
Part 1- Raspberry Pi Wi-Fi setup and packet delivery

➤ Graphical Flowchart of Raspberry Pi Program



(B.) Raspberry Pi Program Flowchart

➤ Graphical Flowchart of ESP8266 Program



(C.) ESP8266 Program Flowchart

➤ **Part 1: Raspberry Pi Wi-Fi Setup and Packet Delivery**

- code implements a sophisticated **IoT system** where ESP8266 devices measure light intensity (LDR values) and communicate over a network to determine a "master" device. The system utilizes a **UDP broadcast protocol** to enable devices to share sensor data and control status updates, all managed by a central Raspberry Pi server.

Key Components:

1. Wi-Fi and UDP Communication:

- ESP8266 devices and the Raspberry Pi server establish a connection using Wi-Fi credentials.
- Devices broadcast their data using UDP packets, which are received by the Raspberry Pi.

2. Master Device Selection:

- Each ESP8266 reads its LDR value and broadcasts it.
- The Raspberry Pi determines the master device based on the highest LDR value, ensuring only one master exists at any given time.
- The Raspberry Pi updates the LED status to indicate the current master device.

3. LED Indication and LDR Visualization:

- The Raspberry Pi uses **LED bars** to represent LDR intensity levels for the master device. The LED bar graph maps the LDR value to a corresponding number of LEDs turned on, providing a visual representation.
- An SPI-controlled LED matrix visualizes the average of LDR values, showing the intensity trend over time.

4. Graphical Output:

- Two graphs are plotted using Matplotlib:
 - A **line graph** of LDR readings over time.
 - A **bar chart** representing the cumulative time each device has been a master.

5. Reset and Logging:

- The system logs all master changes and associated sensor data to a file.
- A physical button on the Raspberry Pi triggers a system reset, clearing the graphs and resetting devices.

6. Real-time Operations:

- The system supports dynamic addition/removal of devices.
- Data updates are processed in real time, ensuring live visualization and accurate master selection.

➤ **Part 2: ESP WiFi Setup and Packet Delivery & LED Graph :**

1. Initialization

- The ESP8266 starts by initializing **WiFi, LED pins, and UDP communication.**
- A **unique ID** is assigned to the device for identification within the network.
- The device is marked as configured and ready for operation.

2. Periodic Task Management

- **Tickers** are initialized to manage periodic tasks, such as sensor data reading and communication.

3. Main Loop

- The main loop handles two core processes:
 - **LDR Sensor Data Reading:** Continuously reads light intensity values.
 - **Processing Incoming UDP Packets:** Receives and processes data packets from other devices.

4. Configuration Check

- On receiving a packet, the device checks its **configuration status**:
 - If **not configured**, it broadcasts its data.
 - If **configured**, it sends an **online notification** and evaluates sensor readings to determine if it should act as the **Master**.

5. Data Broadcasting

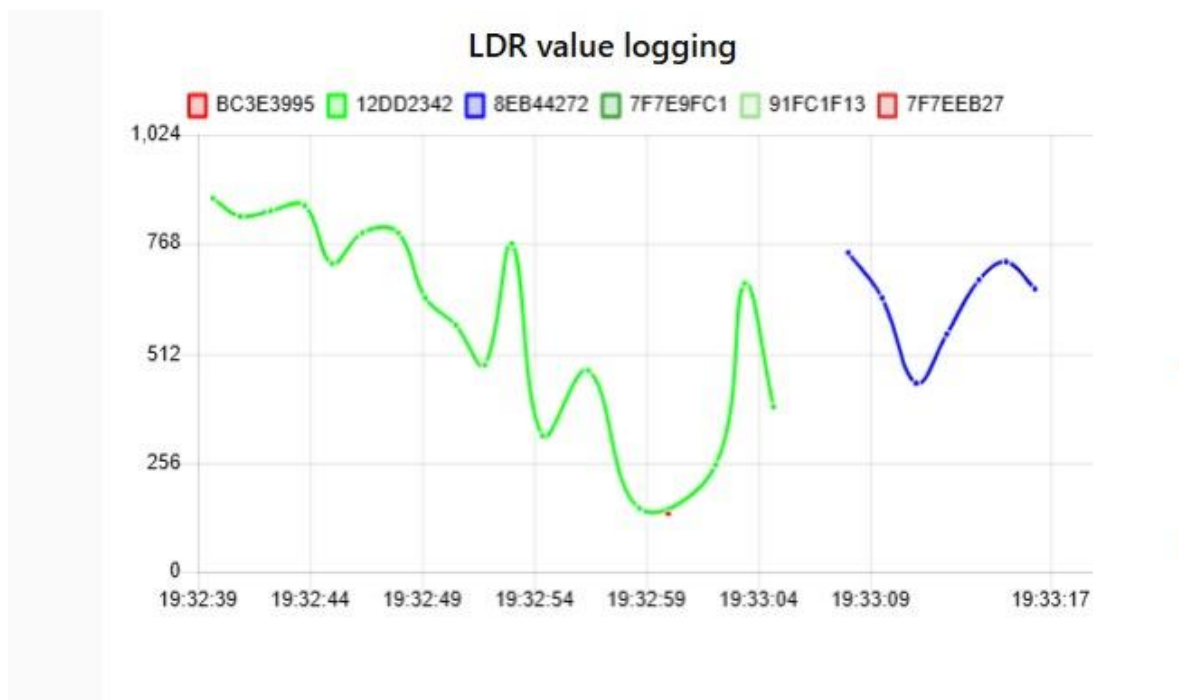
- The device broadcasts its sensor readings or other information to the network using UDP.
- This enables real-time, low-overhead communication between all devices in the IoT system.

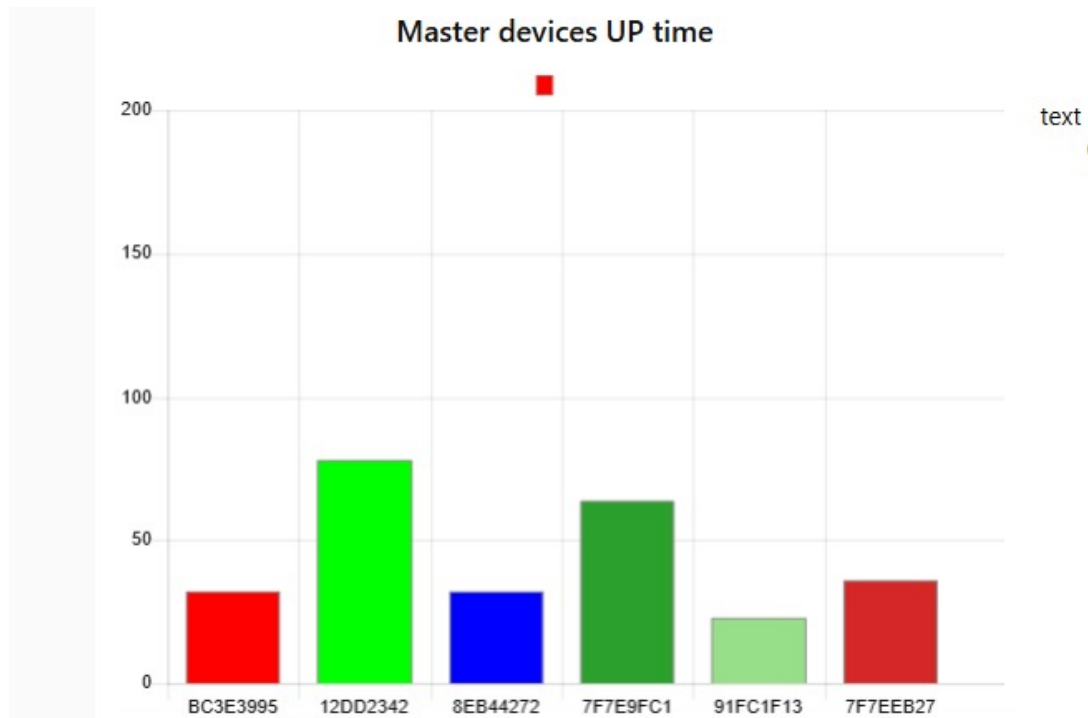
CONFIGURATION SCREENSHOTS

➤ Devices connected to the network:

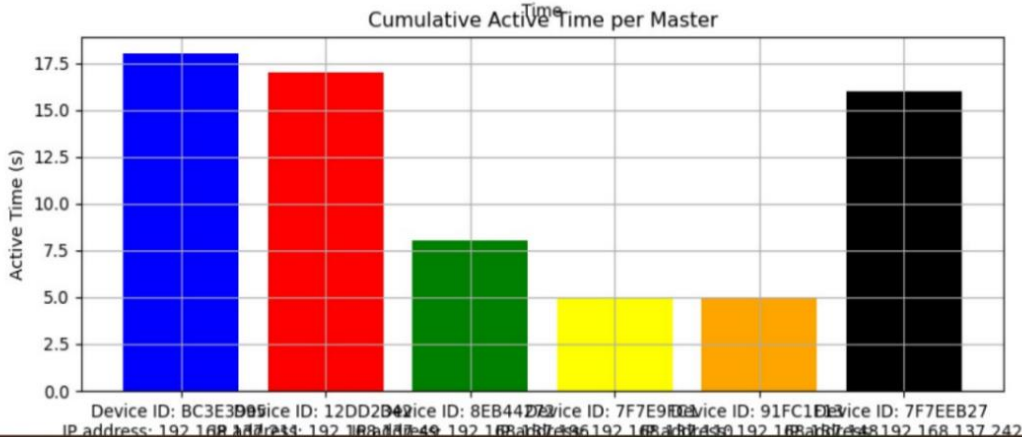
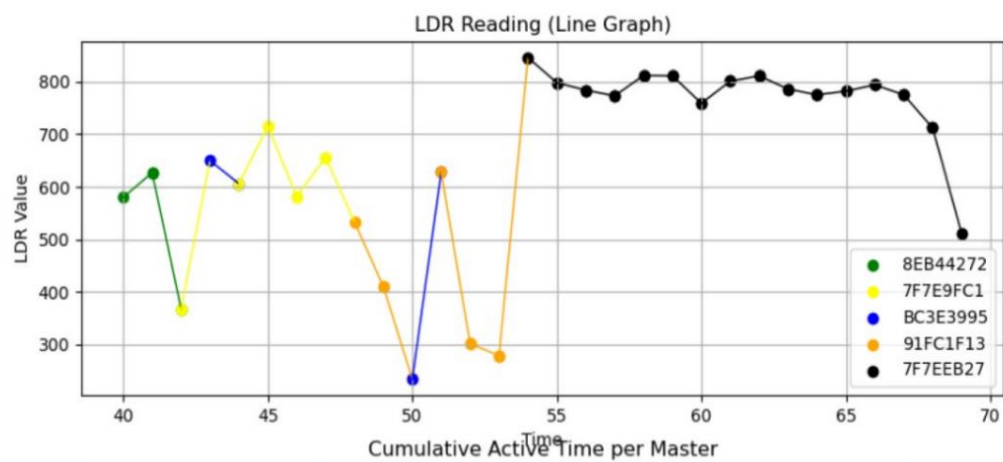
Band:	2.4 GHz	
Devices connected:	7 of 8	
Device name	IP address	Physical address (MAC)
ESP-FC1F13	192.168.137.148	50:02:91:fc:1f:13
ESP-DD2342	192.168.137.49	a4:cf:12:dd:23:42
ESP-B44272	192.168.137.196	84:0d:8e:b4:42:72
ESP-7EEB27	192.168.137.242	5c:cf:7f:7e:eb:27
ESP-7E9FC1	192.168.137.110	5c:cf:7f:7e:9f:c1
raspberrypi	192.168.137.230	2c:cf:67:03:0d:92
ESP-3E3995	192.168.137.211	ec:fa:bc:3e:39:95

➤ Output visible on Node-Red UI-chart along with the photocell data trace & bar chart showing master devices graph.

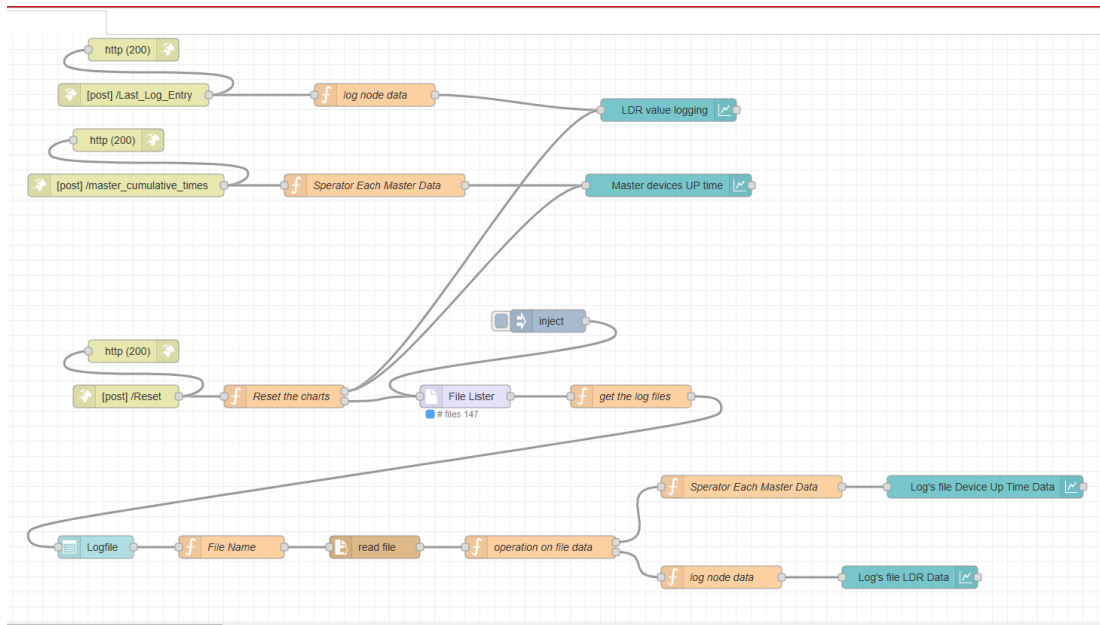




- Raspberry Pi output is visible on the terminal along with the photocell data trace & bar chart showing the master devices graph on the math plot(locally).

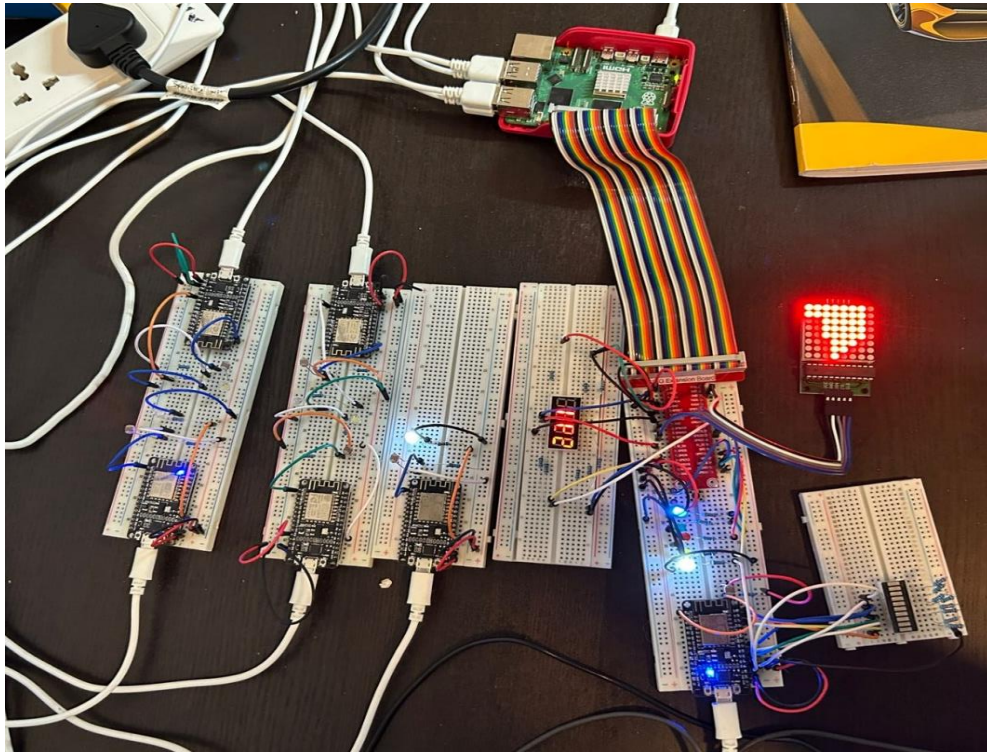


➤ Node-Red flow :

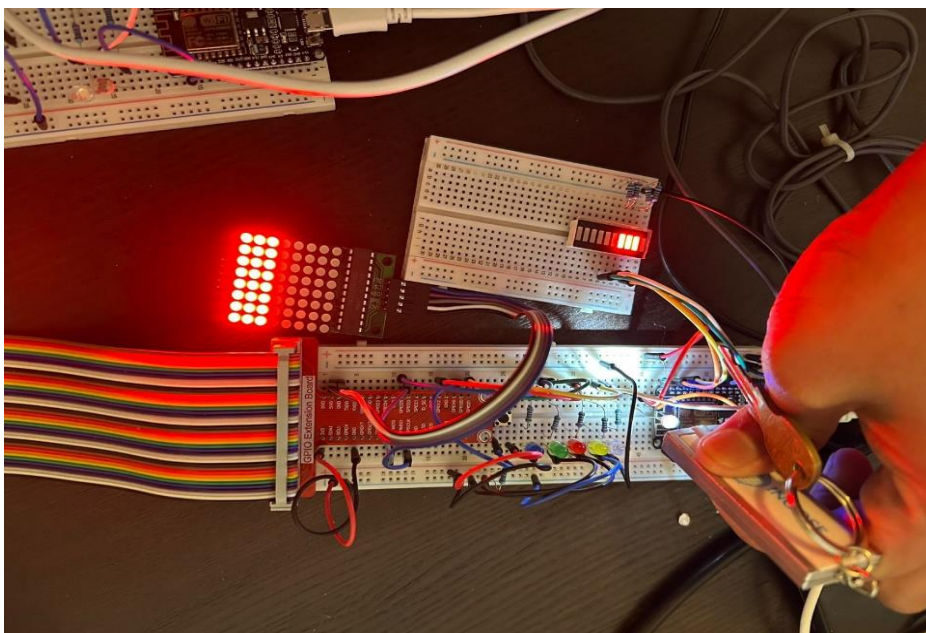


➤ **Working Hardware Picture:**

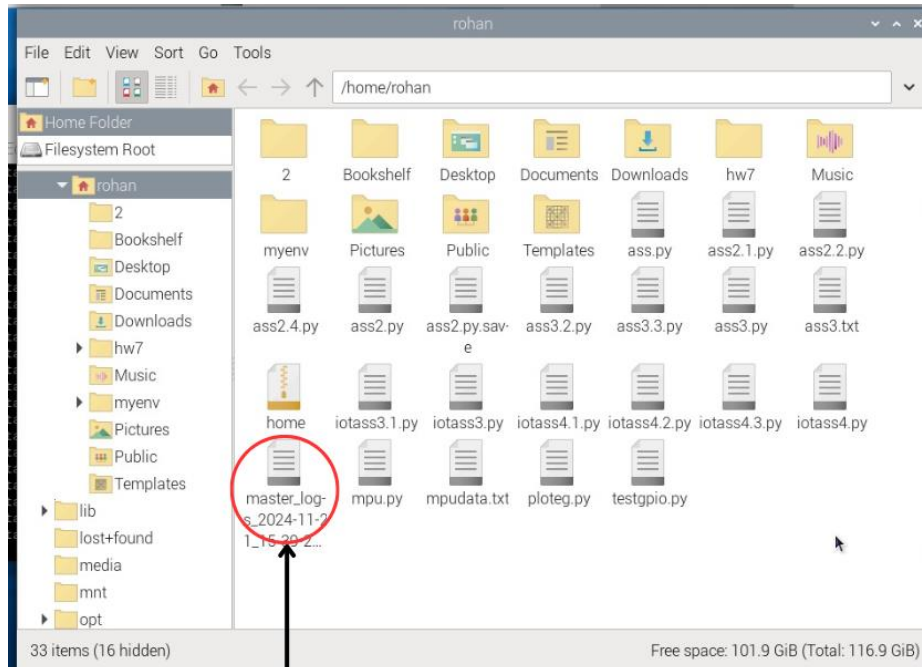
5-ESP8266 CONNECTED IN MASTER & SLAVE CONFIGURATION 4-DIGIT, 7-SEGMENT DISPLAY CONNECTED TO RASPBERRY PI5 TO DISPLAY LAST 3 DIGITS OF THE MASTER'S IP ADDRESS



SHOWING HOW LIGHT INTENSITY IS VARYING THE LED GRAPH CONNECTED TO AN ESP8266 AND ALSO VIEWING THE VARIATION IN LED MATRIX CONNECTED TO RASPBERRY PI5 AT THE SAME TIME



- The logfile is created after pressing the button, thus getting into a reset state, saving the current logfile, and running a new logfile to save data from the master ESP.



**“LOGFILE” CREATED
AFTER THE BUTTON
PRESS**